



Hochschule Aalen
UNIVERSITY OF APPLIED SCIENCES

Machine Learning & Deep Learning

04: DEEP LEARNING

20. April 2026



Overview

- Fundamentals
 - Perceptron
 - Activation Function
 - Bias
- Multilayer feed forward ANN
- Backpropagation
- Overfitting and underfitting

Introduction

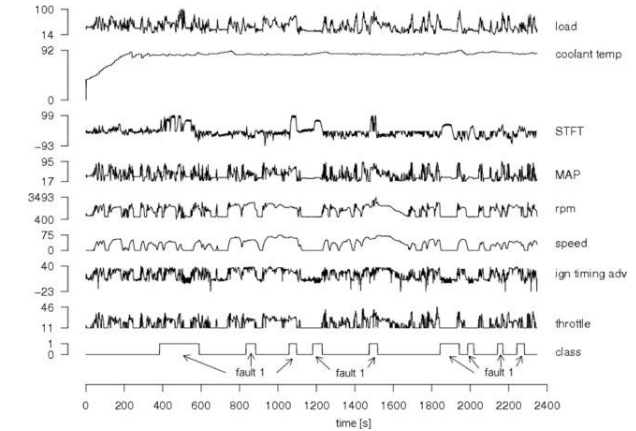
Structured input data = data that can be “numericalized” into a spreadsheet

How do we handle unstructured input data (images, videos, text, audio, ...)

Deep Learning can automatically extract smart representations from raw, unstructured data.

This demolishes the “human bottleneck” for using Machine Learning with unstructured data.

Name	Gender	Height [cm]	Weight [kg]
Peter	M	185	82.5
Maria	F	161	53.0
Emma	F	172	61.3



 **GT Automotive**
2 Elrose Ave, Toronto, ON M9M 2H6, Canada

★★★★★ 3 years ago

They are reliable, honest and operate with integrity. I recommend them to anyone who is looking for some great mechanics.

Tony always takes the time to answer any questions I may have, and provides me with all the options available- from repair to replacement.

As a woman, it can be difficult to find mechanics you can trust with your vehicle, so I'm very happy to have been referred to GT Auto. Tony and John are the best!

👍 1 🔗 Share

Introduction

Structured and
Unstructured data



Deep Learning

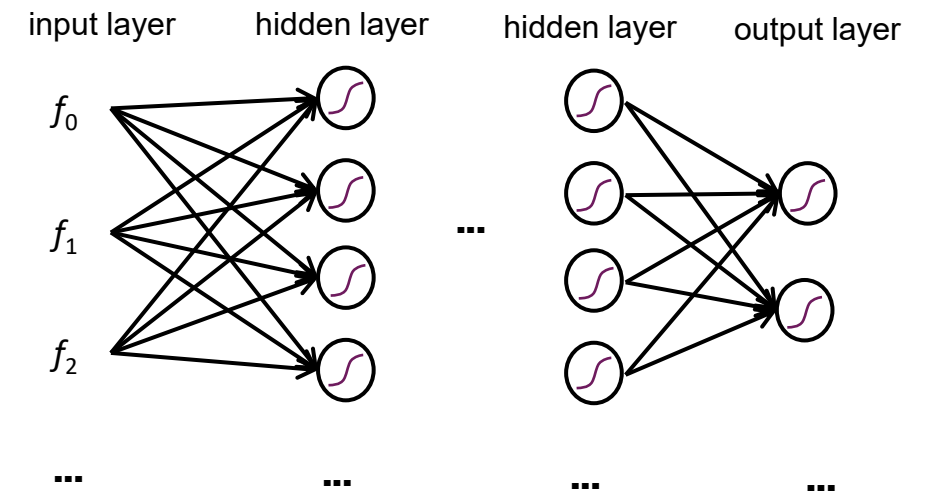
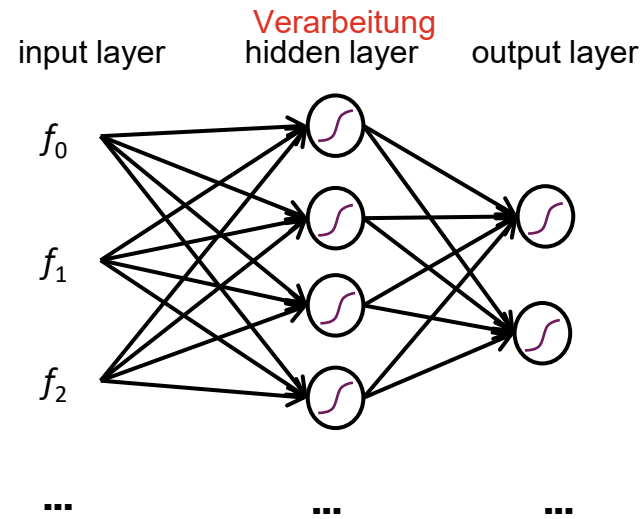


Prediction

It can automatically extract smart representations from raw, unstructured data.

No “human bottleneck” for unstructured data

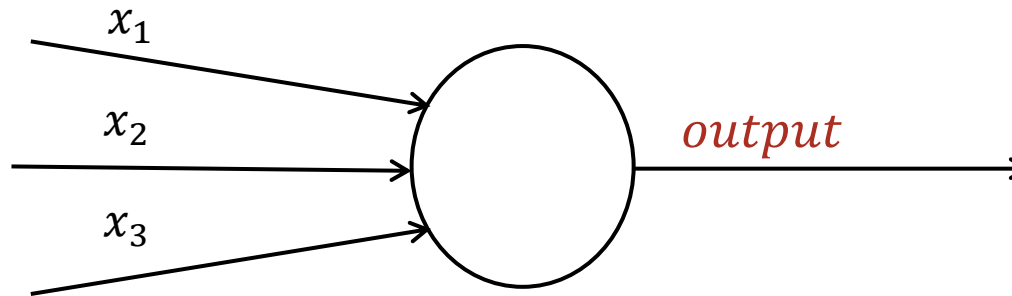
Used on old ML idea:
Neural Networks



Fundamentals

Perceptron

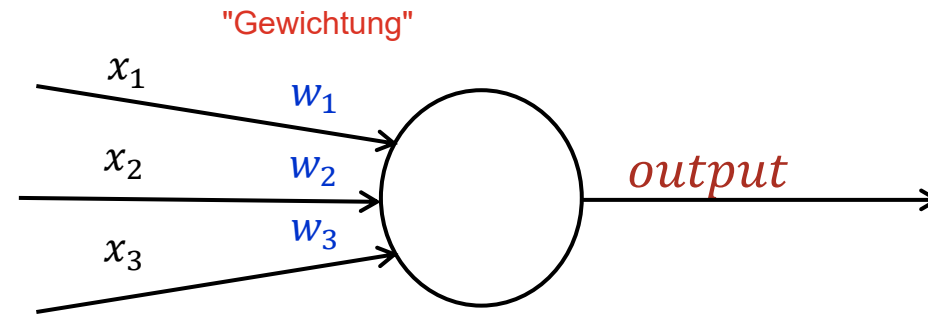
- One trivial artificial neural network consists of one node („neuron“) with multiple inputs and one output.



- Can be used for **linear classification of two classes** („binary classification“).
- Data is passed to the input, one dimension/attribute per input.
 - e.g. input could be height, width, colour value of apples and pears.
- The output is the classification result.
 - e.g. output could be: „it's an apple“ or „it's a pear“.

Fundamentals

Perceptron



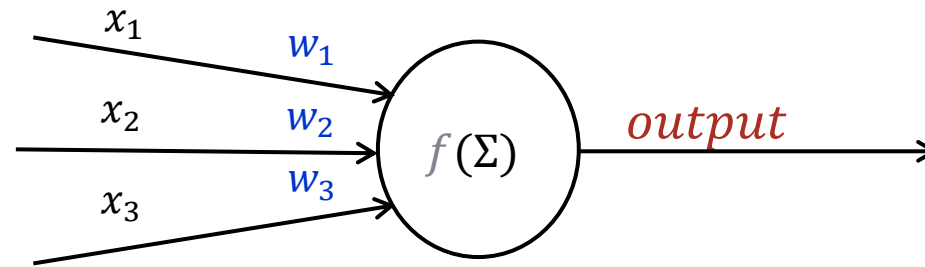
- The inputs of each node $[x]_i$ are weighted with weights w_i that are assigned to each input.
- The weighted inputs are summed up (called „linear combination“):

Jedes weight = individuell

$$w_1 * x_1 + w_2 * x_2 + w_3 * x_3 = \sum_{i=1}^n w_i x_i$$

Fundamentals

Perceptron



- The sum of the weighted inputs is passed as the x-value to a so-called **activation function**
- This **function's value for x**, i.e. $y=f(x)$, is the **node's output** *output* :

Aktivierungswert

$$\text{output}_i = f\left(\sum_{i=1}^n w_i x_i\right)$$

Fundamentals

Perceptron

Calculate the neural networks output for the following inputs:

- input:
 - data point 1: $(2, 1, 4)$ -1.5
 - data point 2: $(1, 2, 1)$ 2.5
- weights: $(0.5, 1.5, -1.0)$

What is the output?

Fundamentals

Activation function

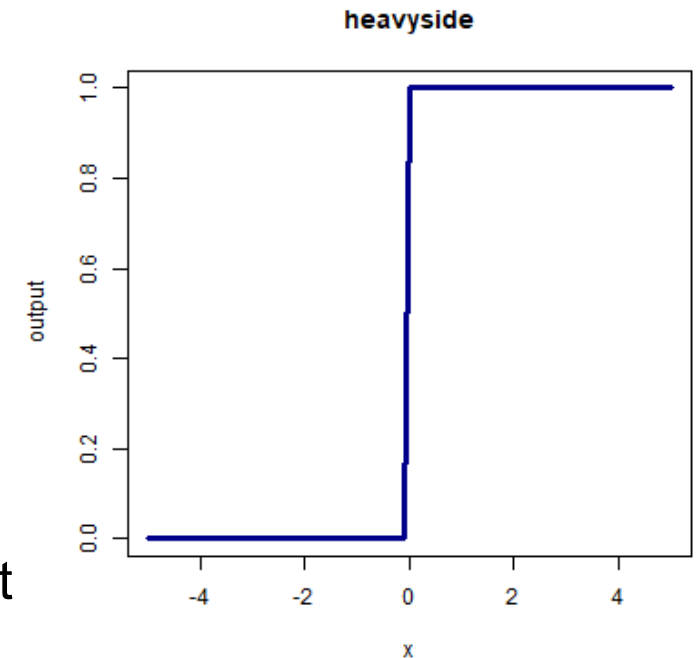
- Transforms a neuron's weighted input into an output, usually introducing nonlinearity (Heavyside is linear) so the network can learn complex patterns

- Heavyside function:

- $$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$$

- $\mathbb{R} \rightarrow [0,1]$

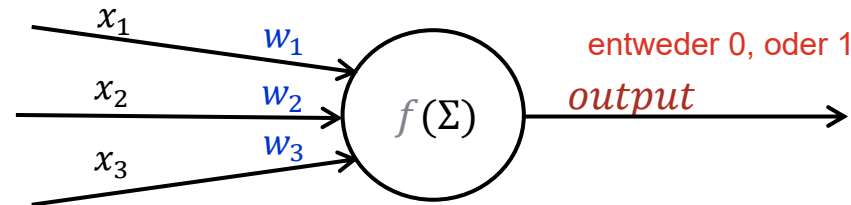
- If the sum of of the network's weighted inputs is >0 , the output
 - Otherwise the output is 0
 - An output of 0 or 1 allows to classify data into two classes based on inputs



Fundamentals

Perceptron

- Can be used for linear classification of two classes („binary classification“)
- Data is passed to the input, one dimension/attribute per input
- Output is the classification result



- The inputs of each node x_i are weighted with w_i and summed up:

$$\sum_{i=1}^n w_i x_i$$

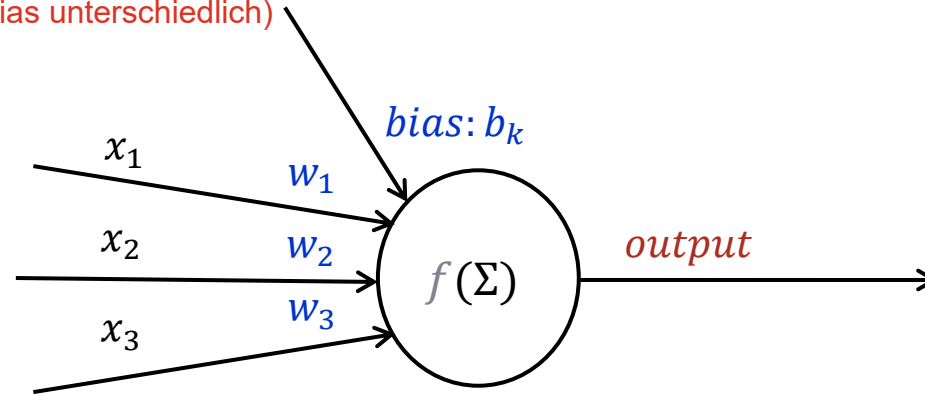
- The sum is passed as to a so-called activation_n function $f()$ (e.g. heavyside) and $f(x)$ is the node's output

$$\text{output}_i = f\left(\sum_{i=1}^n w_i x_i\right)$$

Fundamentals

Bias v.a. bei mehreren nodes (je der Bias unterschiedlich)

Sorgt für Gew. der Nodes;
Unabhängig was Input ist



- In most networks a so-called bias is added to each node
- The bias functions like an offset, it allows to change the output independently of the node's inputs
- Geometrically it is the intercept of a plane described by $w_1 * x_1 + w_2 * x_2 \dots$
- The formula of the weighted sum changes to $w_1 * x_1 + w_2 * x_2 + w_3 * x_3 + b_k = b_k + \sum_{i=1}^n w_i x_i$
- The output is now given by :

$$\text{output} = f\left(b_k + \sum_{i=1}^n w_i x_i\right)$$

Es werden eher einfache Rechenaufgaben

Fundamentals

Vector form Zeigt: Normalerweise weight und features als Vectors angezeigt

- The formula:
$$\text{output} = f(b_k + \sum_{i=1}^n w_i x_i)$$
- Can be formulated in vector form:
$$\text{output} = f(b_k + \mathbf{w}\mathbf{x})$$
- Where $\mathbf{w}$ is the vector $\begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix}$ and $\mathbf{x}$ is the vector $\begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}$.
- Note that $\begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}$ is the inner product (scalar product), which is defined as $\sum_{i=1}^n w_i x_i$.
- In some books the vector $\mathbf{w}$ is transposed, i.e. $\mathbf{w}^T$ (rows to columns or vice versa). We assume have the vectors have correct orientation.

Fundamentals

Perceptron

Calculate the neural networks output for the following inputs:

input:

- data point 1: (2, 1, 4) $-1.5 \rightarrow -1.5 + (-5) = -2 < 0 \rightarrow \text{apple}$
- data point 2: (1, 2, 1) $2.5 \rightarrow 2 - \geq 0 \rightarrow \text{pear}$

output:

- 0 for class 0 (e.g. apple)
- 1 for class 1 (e.g. pear)

weights: (0.5, 1.5, -1.0)

bias term: -0.5

activation function: heavyside

1, wenn $x \geq 0$
0, wenn $x < 0$

Fundamentals

Perceptron - Example (wohl nicht so wichtig)

```
from sklearn.datasets import load_iris
from sklearn.linear_model import Perceptron
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

# load Iris dataset
iris = load_iris()

# Use only setosa and versicolor, and only 2 Features
X = iris.data[:,100, [2, 3]]
y = iris.target[:,100]

# print col names
print("Feature-name:", iris.feature_names[2:4])
print("Classes:", iris.target_names[:2])

# Train/Test-Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42, stratify=y)

# train perceptron
clf = Perceptron(random_state=42)
clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)
```

Weil die ersten 100 nur 2 Klassen sind (das nur mir 2 mögl)

```
# Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nWeights:")
print(clf.coef_)
print("\nBias:")
print(clf.intercept_)
print("\nClassificationreport:")
print(classification_report(y_test, y_pred,
                             target_names=iris.target_names[:2]))
```

Feature-names: ['petal length (cm)', 'petal width (cm)']
Classes: ['setosa' 'versicolor']
Accuracy: 1.0

Weights:
[[0.6 1.7]]

Bias:
[-3.]

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	1.00	1.00	1.00	10
accuracy			1.00	20
macro avg	1.00	1.00	1.00	20
weighted avg	1.00	1.00	1.00	20

Fundamentals

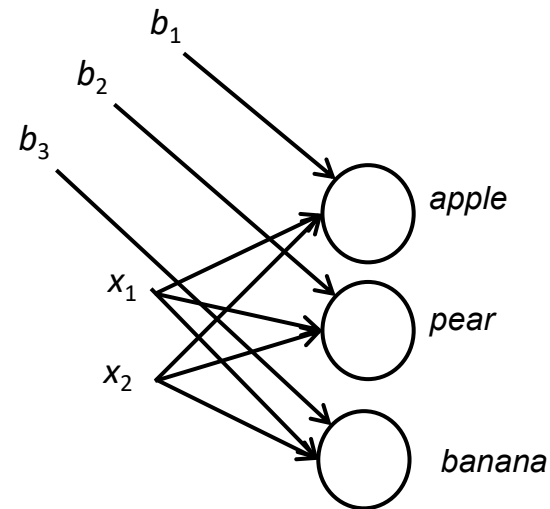
Perceptron for multiple classes

für jede Klasse eigenes Neuron und je dafür die Wahrscheinlichkeit rechnen

- One node can be used to separate two classes
- For more than two classes: one output node per class is used

example:

$f_1 = 7 \text{ cm}$; $f_2 = 6 \text{ cm}$



Gehört wohl zu Softmax

example:

output for *apple* = 1,0,0
output for *pear* = 0,1,0
output for *banana* = 0,0,1

- One output node corresponds to one class, decision is taken by selecting the maximum output value --> Das mit dem größten Value wird verwendet
- The ANN is now a linear classifier for multiple classes

Fundamentals

Activation Functions

- The heavyside activation function $f(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$ was used in early networks.
- Only linear classifiers can be built based on that function.
- A variety of activation functions have been proposed and used since.
- Some of them are:
 - linear
 - logistic (a sigmoid function)
 - tangens hyperbolicus tanh (a sigmoid function)
 - rectifier (the unit is then called called ReLU = rectified linear unit)
andere Layer für input als für output layer
- Different activation functions can be used in layers of the network

Fundamentals

Activation Functions: linear

- linear
 - $\mathbb{R} \rightarrow \mathbb{R}; f(x) = x$

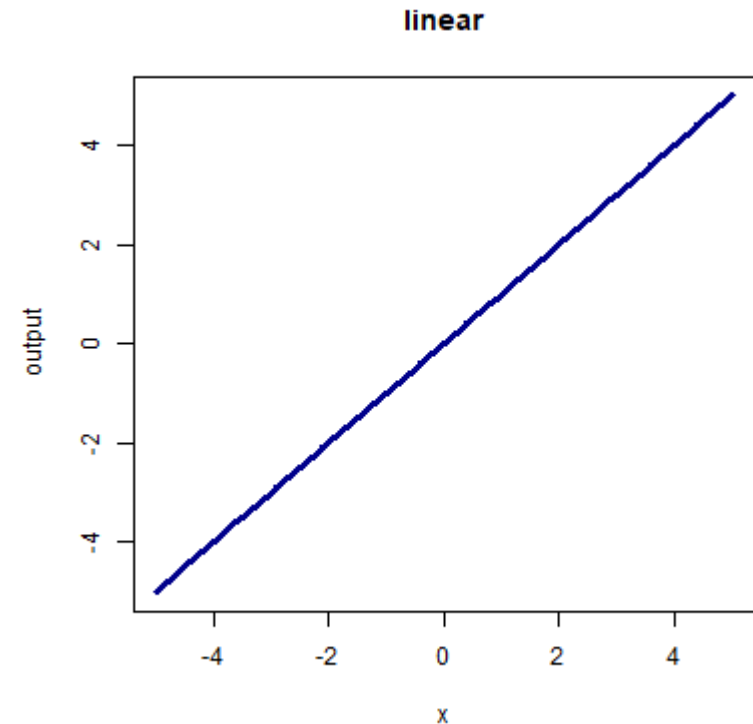
Identity function

Output equals input

Adds no non-linearity

Stacking linear layers still gives a linear model

Commonly used in the output layer for regression



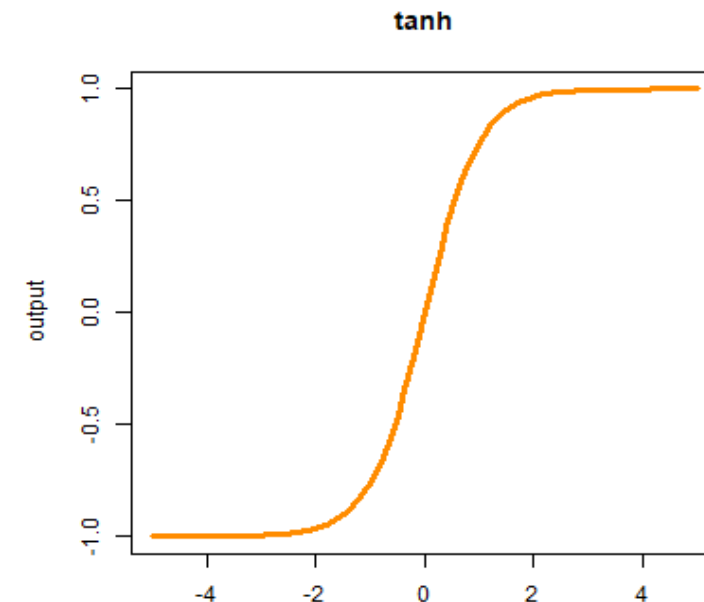
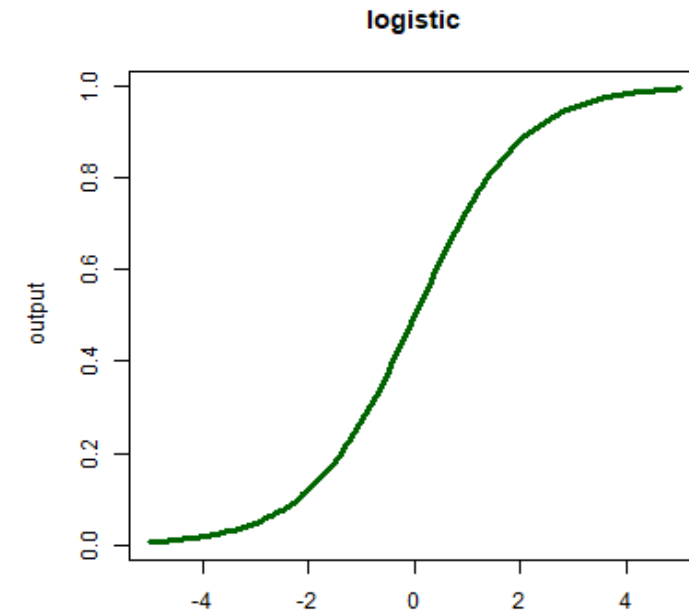
Fundamentals

Activation Functions : sigmoid functions

- sigmoid functions are S-shaped
- logistic:
 - $\mathbb{R} \rightarrow (0,1); f(x) = \frac{1}{1+e^{-x}}$
- tanh:
 - $\mathbb{R} \rightarrow (-1,+1); f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
- sigmoid functions were used many years, and still are used
- they have problems when using many layers number
- the decision function with n parallel nodes is, however, still linear!

Output zw. 0 und 1
Was eben gemapped wird

Output zw. -1 +1
Was eben gemapped wird



Fundamentals

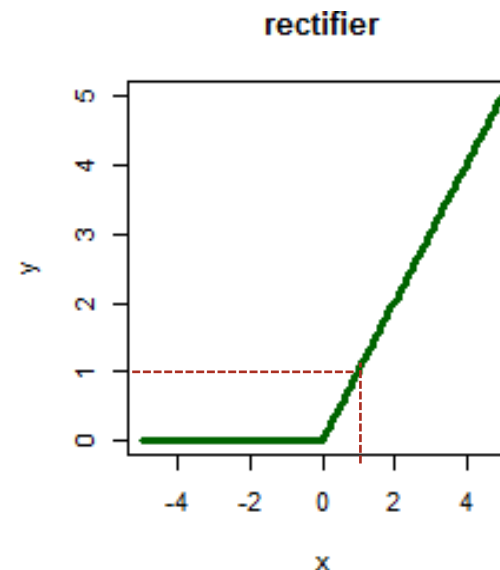
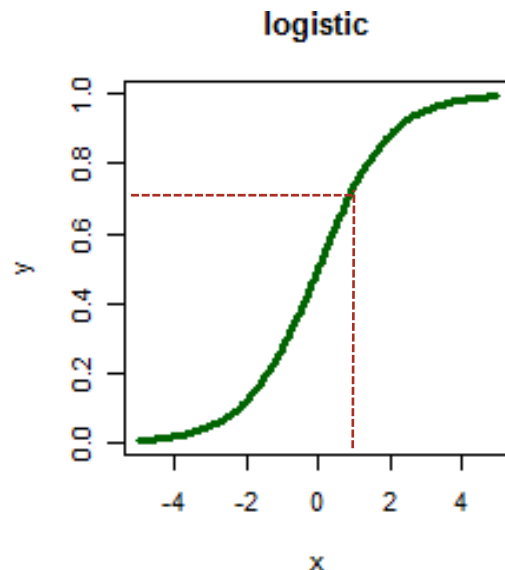
Nennt mir 2-3 verschiedene Activation Funktion / oder 2-3 Sigmoid functions

Activation Functions: ReLU

- commonly used activation functions: instead of logistic function Deep ANNs use rectifier functions (and variants of it)

$$\mathbb{R} \rightarrow [0, \infty]; f(x) = \max(0, x)$$

- the activation function is called rectifier, the unit is then called called ReLU (rectified linear unit)
- (these terms are often used as equivalents also they are not)

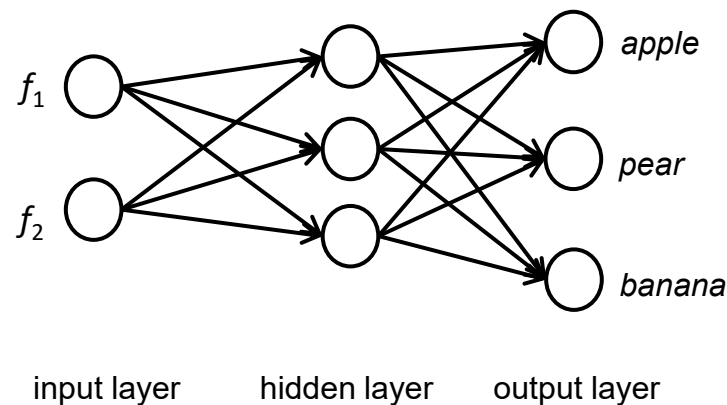


negative Werte werden auf 0 gesetzt
pos. Werte bleiben unverändert

Fundamentals

Activation Functions: Softmax

- For classification problems, the output layer typically uses the softmax activation function
- This function scales the values such that the sum of all output nodes is 1
- The output of one node can then be interpreted as a probability
- Example:
 - output might be at the nodes: apple: 0.7 , pear: 0.2 , banana: 0.1
- so the ANN predicts that the data point is an apple with a probability of 0.7



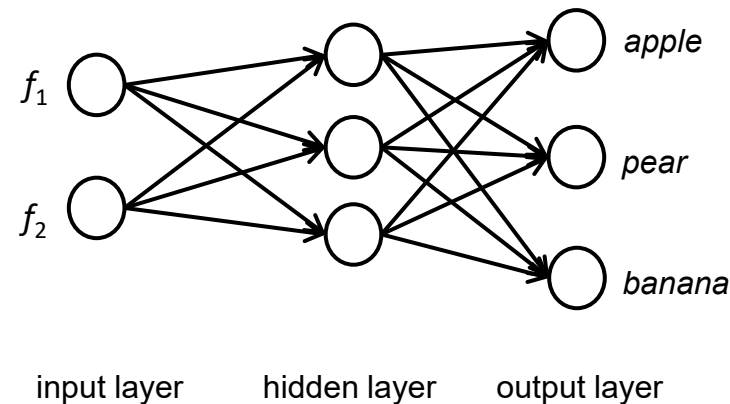
Fundamentals

Activation Function

- Calculate the neural networks output for the following inputs:
- input:
 - data point 1: (2, 1, 4)
 - data point 2: (1, 2, 1)
- weights: (0.5, 1.5, -1.0)
- bias term: -0.5
- activation function: logistic

Multilayer feed forward ANN

- Using multiple layers, a non-linear decision function can be learnt
- Even when the activation functions are simple heavyside functions

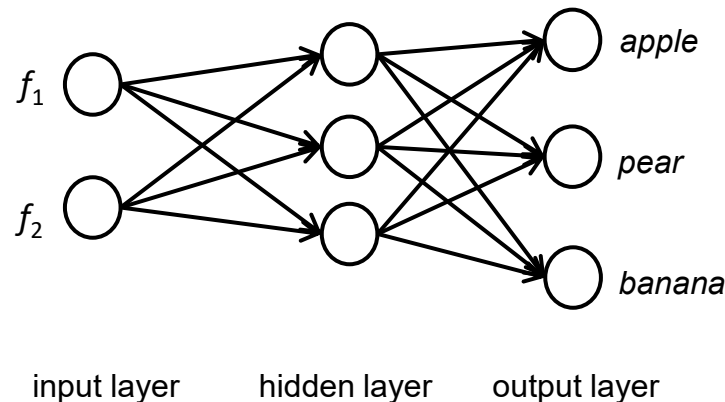


Multilayer feed forward ANN

- Several neurons (nodes) in parallel are one layer
- Several layers are connected sequentially
- Prediction according to outputs at output layer
(note that a two-class problem can be solved with one or two output nodes)
- by connecting the neurons a „network“ is set up: the „artificial neural network“

Input data is passed to the
input layer

Example:
 $f_1 = 7 \text{ cm}$; $f_2 = 6 \text{ cm}$



The **output layer** outputs the
results

Example:
 $\text{apple} = 0.7$
 $\text{pear} = 0.2$
 $\text{banana} = 0.1$
 $\Rightarrow \text{prediction} = \text{max output:}$
 apple

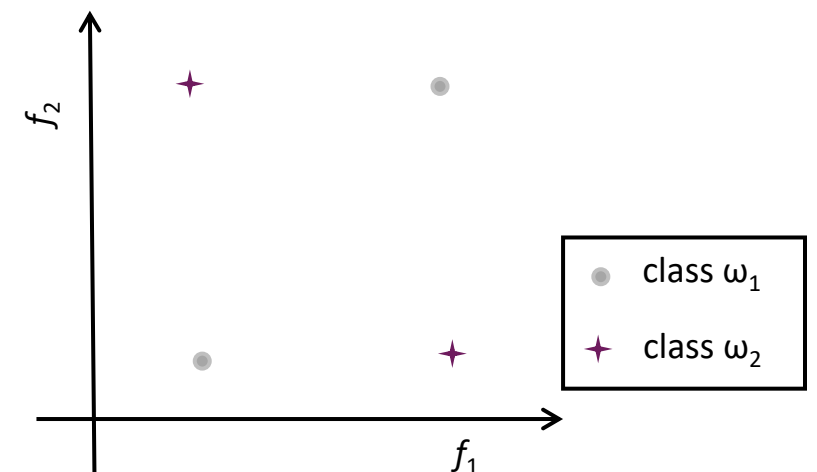
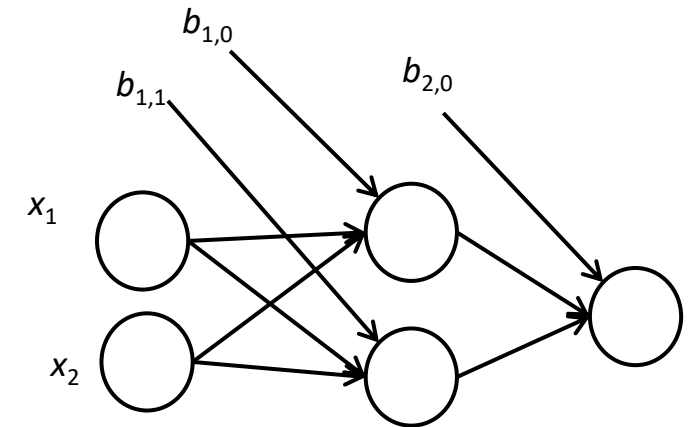
Multilayer feed forward ANN

Showing that a multilayer perceptron can learn non-linear decision functions

Exercise:

- input: $(0,0)$, $(0,1)$, $(1,0)$, $(1,1)$
- weights of hidden layer: $(1, 1)$, $(1, 1)$
- bias of hidden layer: $(-1.5, -0.5)$
- weights of output layer: $(-1, 1)$
- bias of output layer: (-0.5)
- activation function: heavyside function

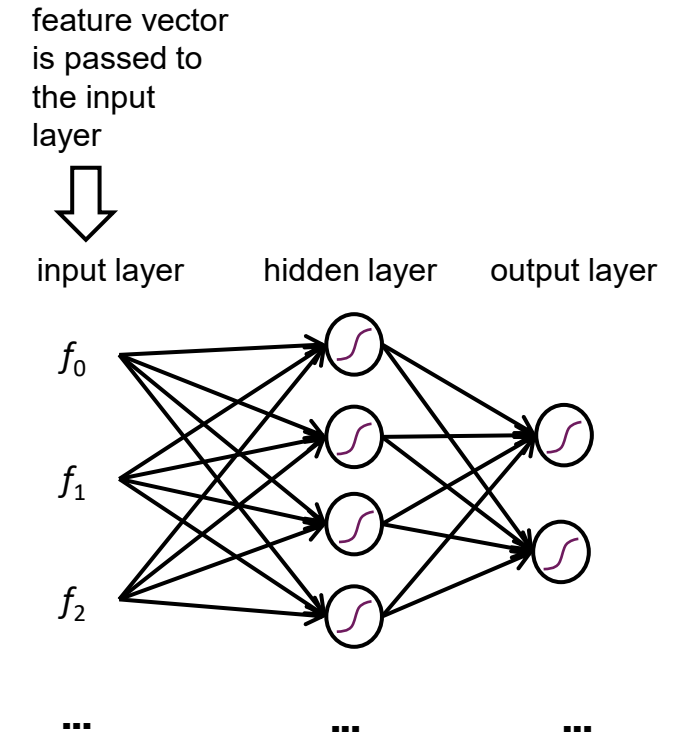
Does it solve the XOR-problem, which for $(0,0)$ is an output of 0 , $(0,1) \Rightarrow 1$, $(1,0) \Rightarrow 1$, $(1,1) \Rightarrow 0$?



Multilayer feed forward ANN

Artificial neural networks (ANNs): overview on one slide

- Idea: mimic the human brain to learn from data
- How it works: pre-define a network of nodes with weighted inputs w
- Training:
 1. classify instances from the training data set
 2. determine the error rate e
 3. adjust the weights w using gradient descent, in order to reduce the error rate
 4. repeat the process to optimize the error rate until the optimum is found (jump to 1.)
- Test: classify an unseen instance, based on the pre-defined topology and the optimized weights w , by passing the instance to the input nodes



Multilayer feed forward ANN

- There is an enormous variety of neural networks, the „traditional“ type that is typically used is as follows:
 - feed-forward multilayer perceptron:
 - „feed-forward“: the information flow is strictly from left to right
 - „multilayer perceptron“: consists of multiple layers of perceptrons
- Activation function: logistic or tanh
- Training using backpropagation
- Artificial neural networks have recently become highly relevant again in the field of deep learning, where so-called deep neural networks are used
- In principle these deep neural networks work similar as shown here, but use some advancements

Multilayer feed forward ANN

Exercise Relevant für Prüfung

An ANN with one hidden layer with 10 nodes shall classify 4 classes based on input data with 5 attributes. (The input layer shall have no weights, no biases and no activation function)

- a) What is the size (number of nodes) of the input layer? 5 (weil 5 input nodes)
- b) What is the size (number of nodes) of the output layer? 4
- c) What is the number of weights of the hidden layer (without biases)? 50 $5 \cdot 10 = 50$
- d) What is the number of biases of the hidden layer? 10 (jeder Node hat bias)
- e) What is the number of connections between hidden layer and output layer? $10 \cdot 4 = 40$
- f) What is the total number of parameters to be tuned? 104 Parameter

Multilayer feed forward ANN

Advantages:

- robustness: robust against noise in the training set
- accuracy: highly flexible decision boundaries
- can be used for different tasks besides classification, e.g. forecasting, regression

Disadvantages:

- scalability: long training period
- interpretability: poor interpretability, the result of the training is a vector/matrix of weights
- network-structure and learning rate has to be pre-redefined
- random parts involved (e.g. initialization of weights)
- global minimum not guaranteed with the standard backpropagation gradient-descent

Artificial neural networks (ANNs)

Backpropagation Bei wie vielen Iterationen war ich richtig , nach wie vielen nicht.

- Backpropagation is used for parameter **tuning**, where the parameters are the weights including the biases
- Backpropagation uses gradient descent (dt. Gradientenabstiegsverfahren)
- The error at the output layer is iteratively reduced
- The weights **are randomly initialized and tuned during the training process**
- The weights **are changed** in a way to reduce the error at the output layer

Artificial neural networks (ANNs)

Backpropagation

- In order to be able to reduce the error, the error needs to be calculated
- This is done using an error function also called loss function
- One common loss function is the „mean sum of squared errors“ MSE:

$$E = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

=> Result davon ist Error

die prediction tatsächliche label

- Where y_i is a vector with the label of the data point i , $\hat{y}_i$ is the networks output (prediction) for data point i , and m is the number of data points
- $\hat{y}_i$ is a function of the weights
- Alternatives: Mean Absolute Error (MAE), Binary/Categorical Cross-Entropy/Log Loss

Artificial neural networks (ANNs)

Backpropagation

- In the loss function, $\mathbf{y}_i$ is required to be a vector in order to be comparable to the networks output this is achieved by transforming the labels using one-hot encoding
- The class label then corresponds to the desired network output

For this example ANN:

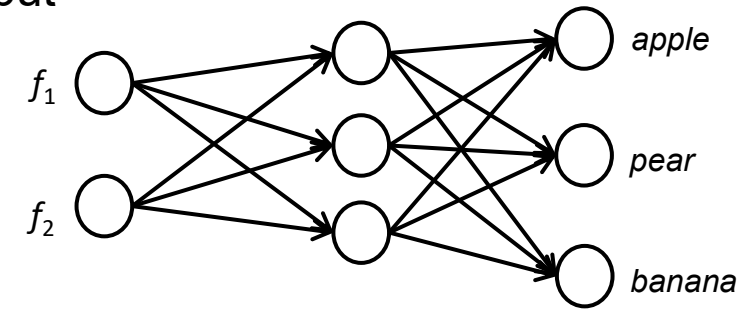
the one-hot encoded labels are:

$$\text{apple} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \text{pear} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \text{banana} = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

predictions (outputs) are floating point numbers,

- e.g. for apple i.e. $\text{apple} = \begin{pmatrix} 0.8 \\ 0.1 \\ 0.1 \end{pmatrix}$ or $\begin{pmatrix} 0.6 \\ 0.2 \\ 0.2 \end{pmatrix}$

the goal is an output that is close to the one-hot encoded label, i.e. for an apple close to $\begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}$



Artificial neural networks (ANNs)

Backpropagation

- When the loss function $E = 0$, the output for all data points corresponds to the class labels
- $E = 0$ is unlikely to be achieved, note that correct predictions are possible without $E = 0$:
- Example:
 - an ANN shall have an output node for apple, pear and banana
 - the output for one data point x_i , which is an apple, shall be $\begin{pmatrix} 0.8 \\ 0.1 \\ 0.1 \end{pmatrix}$
 - decision will be apple
- Exercise: What is the error for this data point ?

$$\begin{aligned} \bullet \quad E_i &= (\hat{y}_i - y_i)^2 \\ &= \text{sum} \left(\begin{pmatrix} 0.8 \\ 0.1 \\ 0.1 \end{pmatrix} - \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \right)^2 = (0.8 - 1)^2 + (0.1 - 0)^2 + (0.1 - 0)^2 = 0.06 \end{aligned}$$

Artificial neural networks (ANNs)

Backpropagation

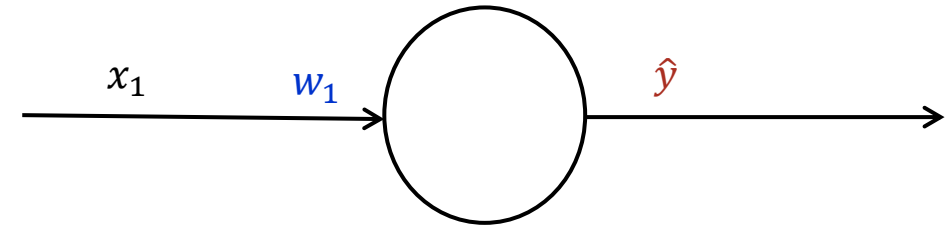
- backpropagation uses gradient descent
- for a very simplified network:
 - one input
 - one output
 - „trained“ with one data point
 - ignoring the activation function (or assuming a linear activation function $y=x$)
- for this simplified example with one data point the loss function

$$E = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

- can be rewritten with $\hat{y}_i = w_1 x_1$:

$$E = (w_1 x_1 - y_1)^2$$

- so the error is a function of the weight, x_1 and y_1 are constant
- we want this error to be minimal



Artificial neural networks (ANNs)

Backpropagation

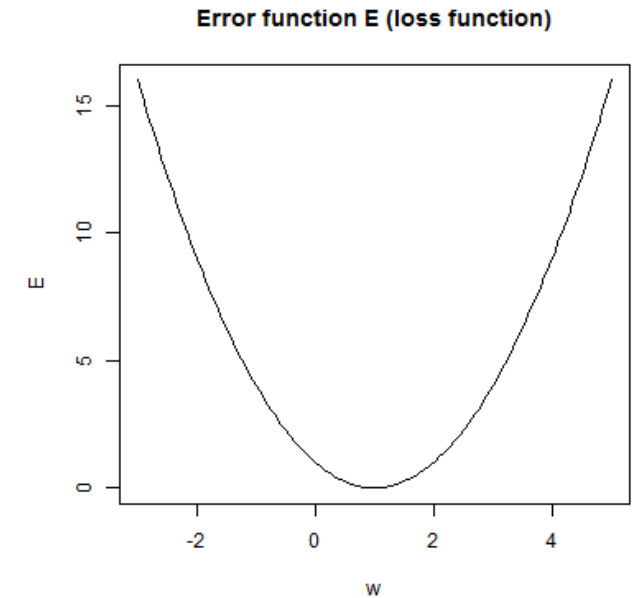
The loss function

$$E = (w_1x_1 - y_1)^2$$

is obviously a quadratic function

We want to find the error function's minimum

We can use the function's derivative and find the value of w_1 , where the function is 0



Artificial neural networks (ANNs)

Backpropagation

Using the chain rule (dt. Kettenregel)

$$f(x) = g(h(x)) \rightarrow f'(x) = g'(h(x)) * h'(x)$$

we find the derivative of

$$E = (w_1 x_1 - y_1)^2$$

to be

$$E' = 2 * (w_1 x_1 - y_1) * x_1$$

during training we do not know all function values, we only know E for the current input x_1 and the current value of weight w_1

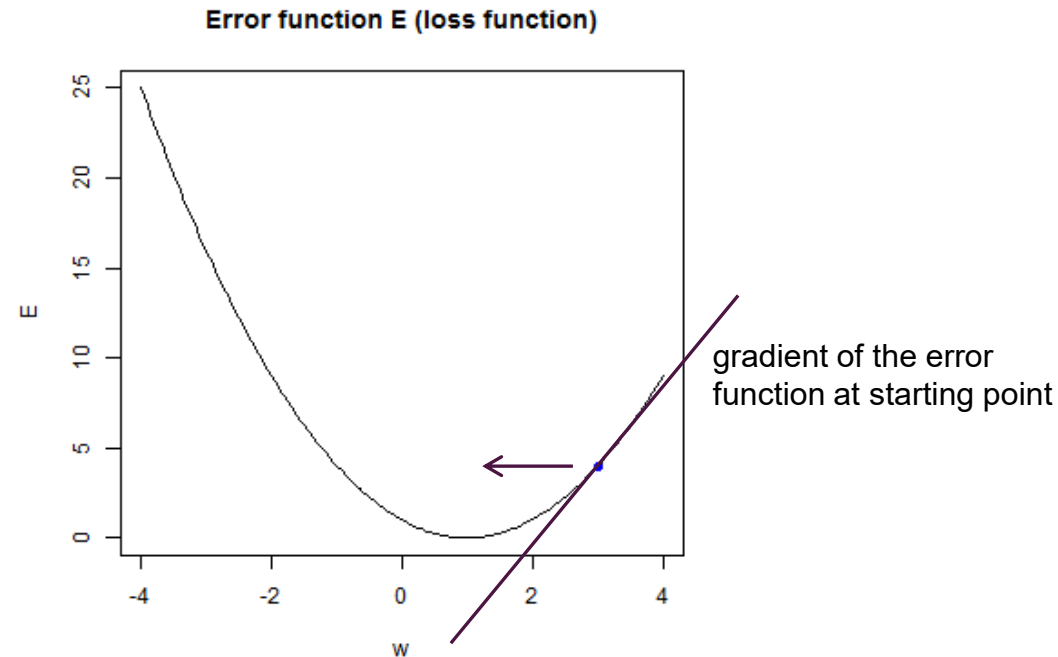
starting with a random w_1 we want to change w_1 in a way that minimizes E

Artificial neural networks (ANNs)

Backpropagation

We know the type of error function (quadratic in our example)

So we can calculate $E(w_1)$ and use the gradient $E'(w_1)$ to find the direction and the extent of the desired change of w



the gradient yields the direction and the extent of the weight update that reduces E

Artificial neural networks (ANNs)

Backpropagation

- To control the extent of the weight update we introduce a learning rate $\eta \in (0,1)$
- The weights are iteratively (step-wise) changed as follows:

$$w_{(\text{step } t+1)} = w_{(\text{step } t)} - \overset{\text{Ableitung}}{\eta * E'(w_{(\text{step } t)})}$$

- The learning rate regulates the trade-off between convergence speed and the stability of the process to converge
- The learning rate is a hyperparameter
- For small networks a constant learning rate of 0.1 is commonly used

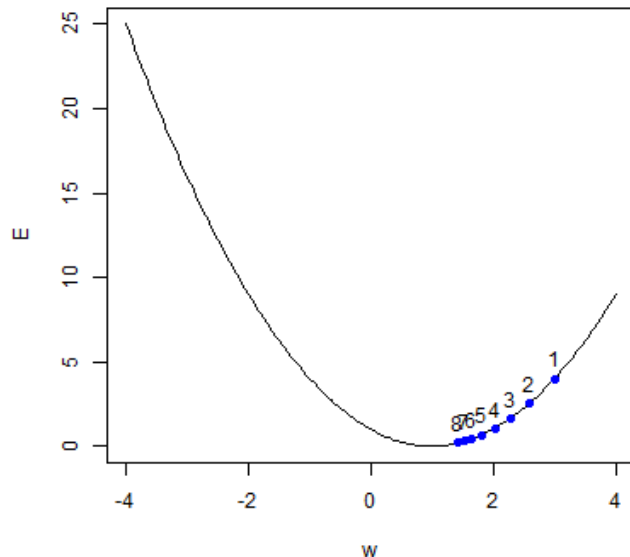
Artificial neural networks (ANNs)

Backpropagation

Für was der Error?

step	w_1	E
1	3	4
2	2.6	2.56
3	2.28	1.63
4	2.02	1.04
5	1.82	0.67
6	1.65	0.43

Gradient descent, learning rate = 0.1



Example: gradient descent

in our example let $x_1 = 1$ and $y_1 = 1$, $\eta = 0.1$

in other words for an input of 1 we want the network's output to be 1

(note that $w_1 = 1$ is the obvious solution)

Starting with a random $w_1 = 3$, the error in the first step is:

$$E(w_1) = (w_1 x_1 - y_1)^2 \\ = (3 * 1 - 1)^2 = 4 \quad 2.56$$

the gradient for $w_1 = 3$ is

$$E'(w_1) = 2 * (w_1 x_1 - y_1) * x_1 \\ = 2 * (3 * 1 - 1) * 1 = 4 \quad 3.2$$

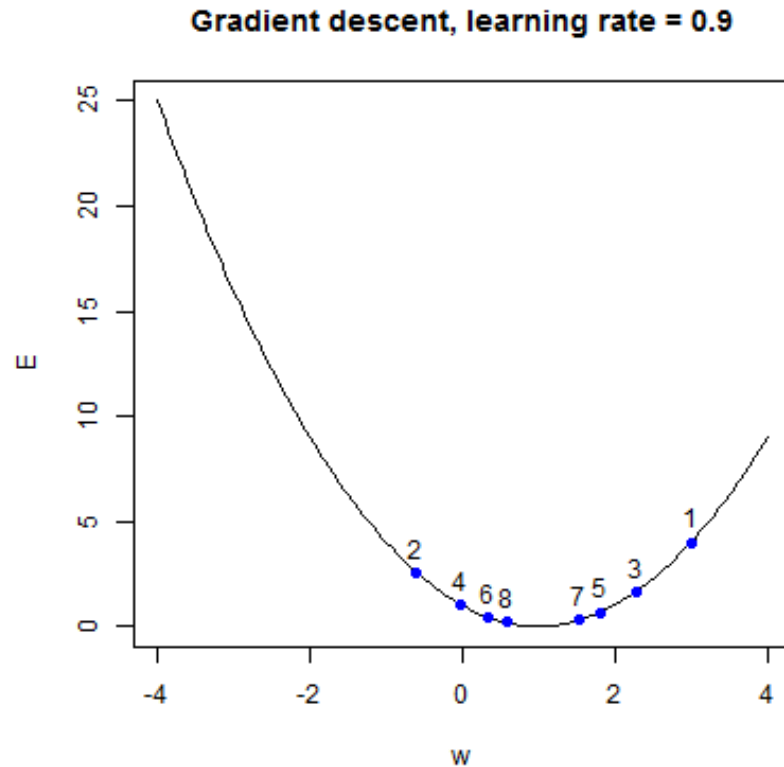
We can calculate the weight of the next step:

$$w_{(step\ t+1)} = w_{(step\ t)} - \eta * E'(w_{(step\ t)}) \\ w_{(step\ t+1)} = 3 - 0.1 * 4 = 2.6 \quad 2.28$$

verhindert große Sprünge

Artificial neural networks (ANNs)

Backpropagation



- higher learning rates lead to faster changes
- step might be so large that minimum is over-stepped
- then the next step will change the weight in the opposite direction
- oscillation is possible
- might diverge instead of converge

Artificial neural networks (ANNs)

Backpropagation

- In the loss function

$$E = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

- $\hat{y}_i$ is the vector of the output layer, e.g. $\begin{pmatrix} 0.8 \\ 0.1 \\ 0.1 \end{pmatrix}$
- each component of this vector is given by the weighted sum of that node's inputs with the activation function applied, i.e. $f(\mathbf{w}\mathbf{x}) = f(\sum_{i=0}^n w_i x_i)$
- this is recursive, i.e. a node's output in layer N depends on the node's inputs, which depend on the nodes' outputs of layer N-1, ...
- in addition to the previous simplified example, the derivative of the activation function is incorporated into the gradient descent steps

Artificial neural networks (ANNs)

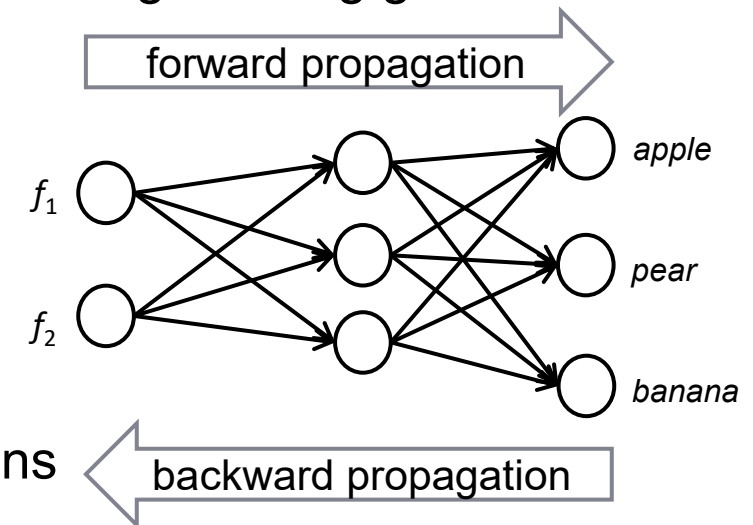
Backpropagation

The weights of all nodes need to be tuned in order to reduce the error. Each iteration of backpropagation has the following two steps:

1. forward propagation: the data is passed to the input layer and the error is calculated at the output layer for all passed data points
2. using gradient descent, the weights of all nodes in all layers are changed using gradient descent

one iteration with all data points is called an epoch

this process is repeated many times, typically thousands of iterations



Artificial neural networks (ANNs)

Backpropagation (Prinzip nur verstehen)

- An ANN has a high number of weights
 - so the function to be optimized is not as simple as shown in the example for one weight by the quadratic function
 - it is a function of very many variables
- The process, however, is the same:
 - the function's gradient is calculated in order to determine the direction and extent the of the weight update
 - instead of the derivative, the partial derivatives are used
- One can imagine that process for two variables as a skate bowl, where we roll a ball and we want it to stop at the minimum:



Artificial neural networks (ANNs)

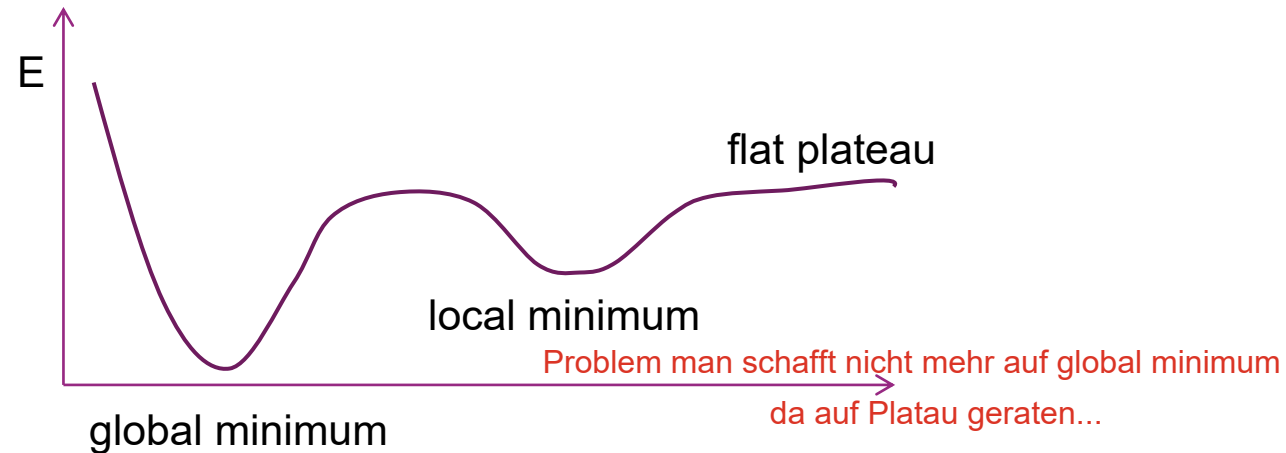
Backpropagation

- Backpropagation does **not guarantee to find the global minimum** of the **error function**, it is ^{nicht immer quadratisch} possible to get stuck at a local minimum. i.e. it is not guaranteed to have the „optimal“ neural network after training
- Common problems:
 - running into local minima
 - overstepping of global minimum
 - oscillation around the minimum
 - slow or no convergence due to flat plateaus (the gradient is close to 0 at flat plateaus)
- although these problems exist, solutions have been found, so that back propagation works well in practice
 - one simple approach: repeating the process multiple times ^{weights werden jedes mal anderes gewichtet}
 - good choice of activation function
 - advanced versions of back propagation

Artificial neural networks (ANNs)

Backpropagation

- A fictitious error function for one weight:



- In reality it is a function of many weights (easily thousands of weights)
- In the plot it can be seen that, depending on the random starting point and the learning rate, it is possible to arrive at the local minimum
- Also slow convergence is possible if the starting point is in the region of the flat plateau

Artificial neural networks (ANNs)

Backpropagation

- There are various options when to update the weights (there are many different names for these variants):
- epoch updating = batch updating:
 - the whole training set is passed to the ANN
 - the mean error is calculated (summing up the errors and dividing by the number of data points)
 - the weights are updated
 - for a large training set, this process is quite slow
- **mini-batch updating** (=stochastic gradient descent (SGD) with batch size > 1):
 - a small number of random samples (e.g. 50) are taken from the training set
 - the mean error is calculated (summing up the errors and dividing by the number of data points)
 - the weights are updated
 - faster update of weights, but updates depend only on a small subset of the data
 - common approach for Deep Learning

Artificial neural networks (ANNs)

Backpropagation

- There are various options when to update the weights (there are many different names for these variants):
- **case updating** = online updating (=stochastic gradient descent (SGD) with batch size = 1):
 - **one data point** is **randomly** selected from the training set
 - **error** is **calculated** and the **weights** are **updated**
 - fast update of weights, **possibly not robust** (weights might “jump”)

Artificial neural networks (ANNs)

Backpropagation

Termination of backpropagation

- Note:
 - the exact minimum is unlikely to be found (the process converges towards the minimum)
 - the minimum is not likely to be 0
- So backpropagation needs to be stopped based on some condition
- Some common termination conditions are:
 - if a pre-defined number of epochs has been completed
 - if the error is below some pre-defined threshold
 - if the improvement of the error is below some pre-defined threshold
 - if the weight updates are below some pre-defined threshold

Artificial neural networks (ANNs)

Optimization

- Deep Learning requires a large training set
- Epoch learning, i.e. adapting the weights after having seen all data points, would not converge, since it would take very long *nicht sinnvoll*
- Mini-batch updating is used:
 - mini-batch updating (=stochastic gradient descent (SGD) with batch size > 1):
 - a small number of random samples (e.g. 50) are taken from the training set
 - the mean error is calculated (summing up the errors and dividing by the number of data points)
 - the weights are updated
 - faster update of weights, but updates depend only on a small subset of the data
 - common approach for Deep Learning

Artificial neural networks (ANNs)

Optimization

- **Momentum optimization:** bei kleinen batches --> können errors spiken --> weil outliers viel mehr gewichten
 - For smaller batch sizes (mini-batch updating or case updating), the change of weights might be instable
 - A momentum term can be introduced, that incorporates the previous weight update into the current weight update
 - i.e. if the weights were rapidly updated in positive direction and the current batch indicates an update in negative direction, this change of direction is smoothed

$$w_{(step\ t+1)} = w_{(step\ t)} - (\beta m + \eta E'(w_{(step\ t)}))$$

- Where m is a vector with the previous gradients
- And β is a hyperparameter between 0 and 1 (0 = no momentum, 1 = maximal momentum). A common value for β is 0.9.

Artificial neural networks (ANNs)

Optimization

- ^{Trainingprozess in kleinere Steps, v.a. in den letzteren Schritten} Learning rate decay / adaptive learning rate:
 - The learning rate η can be gradually decreased during the training process
 - This allows to take large steps at the beginning and finer steps towards the end of the training process
 - Enhanced versions of backpropagation that use an adaptive learning rate are AdaGrad and RMSProp
- Learning rate decay / adaptive learning rate + momentum optimization:
 - Momentum optimization and adaptive learning rate is combined in ADAM (Adaptive momentum estimation)

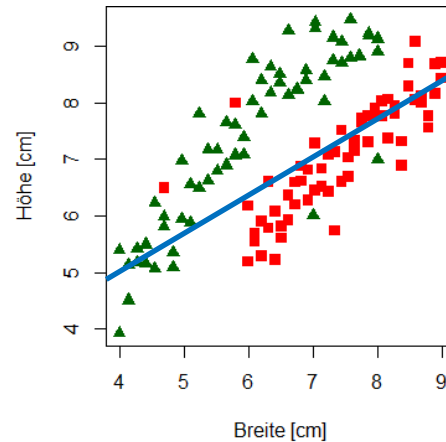
Artificial neural networks (ANNs)

Overfitting and underfitting

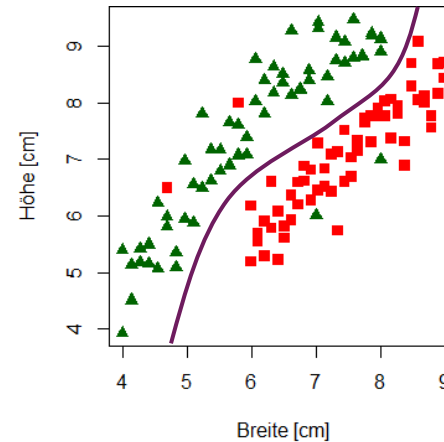
- We do not want the optimal solution on the training set.
- We want an optimal solution on unseen data!
- In general:
 - overfitting: the classifier learns the specifics of the training set, but does not generalize well
 - underfitting: the classifier does not learn the training set well enough (e.g. because the decision functions are not flexible enough).
- For ANNs:
 - overfitting: ANNs can have highly flexible decision functions, so overfitting is a common problem
 - underfitting: underfitting is not as likely to occur
 - it can be shown for very small ANNs, e.g. perceptron with only one layer
 - it occurs if the data set is more complex than the ANNs ability to model complex decision functions
 - or if the ANN has not been trained long enough

Artificial neural networks (ANNs)

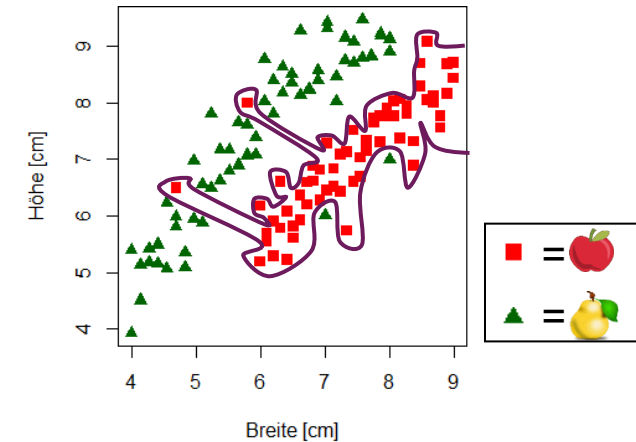
Overfitting and underfitting Woran erkennt man over- und underfitting? Diese unterscheiden, definieren (Prüfung)



„underfitting“:
error on training set: high
error on test set: high



good model:
error on training set: low
error on test set: low



„overfitting“:
error on training set: very low
error on test set: high

Overfitting of ANNs:

- ANNs can be „universal function approximators“, i.e. they can learn any mapping from input to output
- this can easily lead to overfitting
- the more complex the model is (e.g. Deep ANNs) the more flexible the decision functions are, i.e. the higher the chance for overfitting

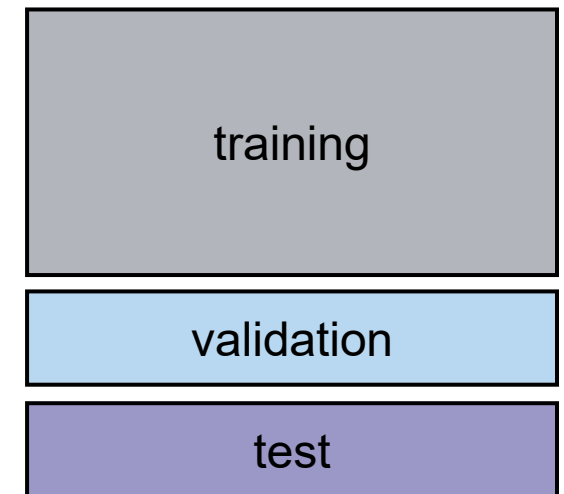
Artificial neural networks (ANNs)

Overfitting and underfitting

In order to monitor the process during training, the ANNs error in the current step is calculated not only on the training set, but on a so-called validation set

this is an approach to try to detect overfitting

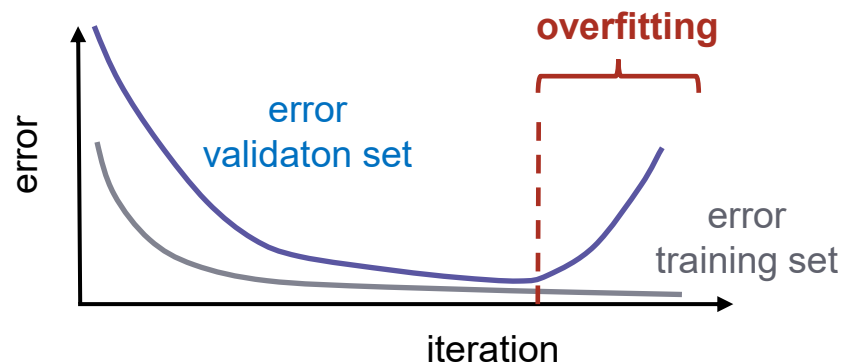
- This validation set is split from the initial training set
- During a step multiple validation sets can be extracted using “cross-validation”
- Note: a „blind test set“ should still be kept, i.e. validation set is not the same as the test set
- The blind test set is used after the training process



Artificial neural networks (ANNs)

Overfitting and underfitting

- Observation:
 - error on training set (train error, train loss) decreases
 - error on test set (test error, test loss) decreases up to some point after that point, the error on the test set increases => overfitting

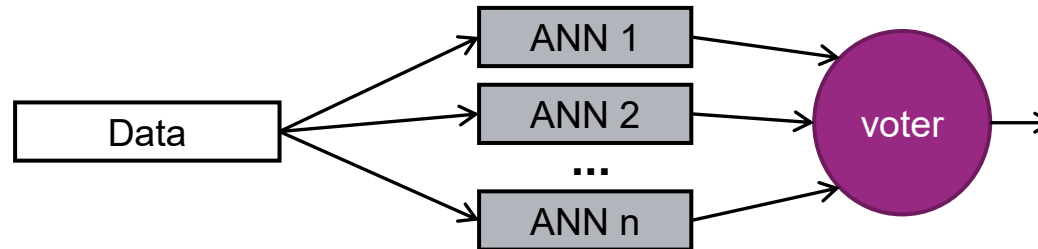


- Approach: stop training when overfitting starts („early stopping“)
 - this is not done on test set but on the validation set
 - stop training when error on validation set (validation error, validation loss) increases
 - since the errors are not as stable as shown here, the errors **are monitored** during training and training is stopped after a „stable“ increase of the validation error

Artificial neural networks (ANNs)

Overfitting and underfitting

- ^{Overfitting umgehen} **Ensemble** - Combining multiple models
- Observation: All models have strengths and weaknesses; the type of overfitting depends on the model employed
- Idea: Combination of multiple different models
- Options:
 - ANNs use differing architectures: number of layers, number of neurons, etc.
 - Identical ANNs are trained on different datasets/parts of the dataset
 - Thus different weights w_i



Results of different ANNs are combined by „voting“ (classification)

Example:

ANN 1: „Apple“
ANN 2: „Pear“
ANN 3: „Apple“

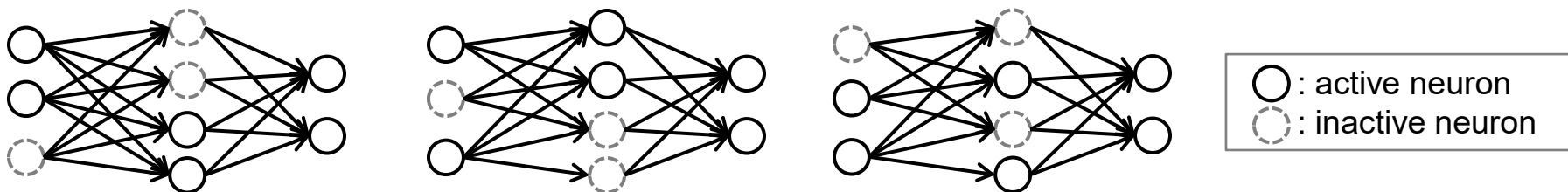
} = „Apple“

Artificial neural networks (ANNs)

Overfitting and underfitting

Dropout Deaktivierung von n Neuronen, damit Modell nur mehr versuchen muss zu generalisieren, weil dieser sich nicht immer pro Epoche jedes Features u.ä. anschaut. Zur Vermeidung, dass pro Epoche immer die selben Neuronen betrachtet werden.

- Observation: The method of „ensembling“ works well, but it is resource intense and needs larger training datasets
- Idea: Use different characteristics of the same ANN
- How does it work:
 - Per sample („data instance“), random deactivation („dropout“) of neurons and their corresponding connections.
 - Deactivation of neurons: e.g. $p=20\%$ in the input layer, $p=50\%$ in the hidden layer
 - All neurons are active in the testing phase



Summary

- Deep learning automatically extracts features from unstructured data.
- Perceptron = the simplest artificial neuron for linear classification.
- Weights + bias + activation function determine the output.
- Nonlinear activations (Sigmoid, tanh or ReLU) enable complex decision boundaries.
- Multilayer ANNs can also solve nonlinear problems.
- Backpropagation optimizes weights via gradient descent.
- Overfitting is reduced, for example, with early stopping and dropout.

Control Questions

1. Why is a bias useful?
2. How does backpropagation work?
3. Why is a validation dataset important during training, even though there is also a test dataset?
4. Why are nonlinear activation functions crucial to the success of deep neural networks?
5. What are the risks of using a learning rate that is too high?